

207. 课程表

Category	Difficulty	Likes	Dislikes	ContestSlug	ProblemIndex	Score
algorithms	Medium	(Not Available)	(Not Available)	-	-	0

- **Tags:** 深度优先搜索, 广度优先搜索, 图, 拓扑排序
- **Companies:** (Not Available)

你这个学期必须选修 `numCourses` 门课程，记为 `0` 到 `numCourses - 1`。

在选修某些课程之前需要一些先修课程。先修课程按数组 `prerequisites` 给出，其中 `prerequisites[i] = [ai, bi]`，表示如果要学习课程 `ai` 则 **必须** 先学习课程 `bi`。

- 例如，先修课程对 `[0, 1]` 表示：想要学习课程 `0`，你需要先完成课程 `1`。

请你判断是否可能完成所有课程的学习？如果可以，返回 `true`；否则，返回 `false`。

示例 1:

```
1 输入: numCourses = 2, prerequisites = [[1,0]]
2 输出: true
3 解释: 总共有 2 门课程。学习课程 1 之前，你需要完成课程 0 。这是可能的。
```

示例 2:

```
1 输入: numCourses = 2, prerequisites = [[1,0],[0,1]]
2 输出: false
3 解释: 总共有 2 门课程。学习课程 1 之前，你需要先完成课程 0；并且学习课程 0 之前，你还应先完成课程 1。这是不可能的。
```

提示:

- `1 <= numCourses <= 2000`
- `0 <= prerequisites.length <= 5000`
- `prerequisites[i].length == 2`
- `0 <= ai, bi < numCourses`
- `prerequisites[i]` 中的所有课程对 **互不相同**

[Discussion](#) | [Solution](#)

解决方法

1. 问题理解

目标： 判断给定的课程依赖关系下，是否能完成所有课程的学习。

输入：

- `numCourses`: 课程总数，编号 0 到 `numCourses - 1`。
- `prerequisites`: 一个二维数组，`[ai, bi]` 表示学习 `ai` 必须先学习 `bi`。

输出： `true` 如果可以完成所有课程，`false` 如果存在循环依赖导致无法完成。

核心问题： 检测课程依赖关系中是否存在环。如果存在环（例如 A 依赖 B，B 依赖 C，C 依赖 A），则无法完成所有课程。

2. 思路分析

这个问题本质上是检测一个**有向图**中是否存在环。课程可以看作图中的节点，先修关系 `[ai, bi]` 可以看作一条从 `bi` 指向 `ai` 的有向边（表示学完 `bi` 才能学 `ai`）。

方法一：拓扑排序 (基于 BFS / Kahn 算法)

拓扑排序是针对有向无环图 (DAG) 的排序算法。如果一个图可以进行拓扑排序，则它一定是 DAG，即无环。

1. 构建图和入度数组：

- 创建一个邻接表 `adj` (例如，字典或列表数组) 来表示图，`adj[i]` 存储课程 `i` 指向的所有后续课程。
- 创建一个入度数组 `in_degree`，`in_degree[i]` 存储指向课程 `i` 的边的数量（即课程 `i` 的先修课程数量）。
- 遍历 `prerequisites`：对于每个 `[ai, bi]`，表示有一条从 `bi` 到 `ai` 的边。
 - 将 `ai` 添加到 `adj[bi]` 的邻接列表中。
 - 增加 `ai` 的入度：`in_degree[ai] += 1`。

2. 初始化队列：

- 创建一个队列 `queue`。
- 将所有入度为 0 的课程（没有先修课程要求的课程）加入队列。

3. BFS 过程：

- 初始化一个计数器 `count = 0`，用于记录已完成（已成功拓扑排序）的课程数量。
- 当队列 `queue` 不为空时：
 - 从队列中取出一门课程 `u` (表示这门课可以学习了)。
 - `count += 1`。
 - 遍历 `u` 的所有后续课程 `v` (即 `adj[u]` 中的所有课程)：
 - 将 `v` 的入度减 1：`in_degree[v] -= 1` (表示 `u` 这门先修课已经完成)。
 - 如果 `v` 的入度变为 0，说明 `v` 的所有先修课程都已完成，将 `v` 加入队列。

4. 结果判断：

- BFS 结束后, 比较 `count` 和 `numCourses`。
- 如果 `count == numCourses`, 说明所有课程都成功加入了拓扑排序序列, 图中无环, 可以完成所有课程, 返回 `true`。
- 如果 `count < numCourses`, 说明图中存在环, 导致有些课程的入度永远无法变为 0, 无法加入队列完成学习, 返回 `false`。

方法二: 深度优先搜索 (DFS) 检测环

通过 DFS 遍历图, 并在遍历过程中检测是否存在环。我们需要维护每个节点的状态:

- 0: 未访问 (unvisited)
- 1: 正在访问 (visiting) - 当前 DFS 路径上的节点
- 2: 已访问 (visited) - 从该节点出发的所有路径都已探索完毕, 且未发现环

1. 构建图:

- 创建一个邻接表 `adj`, `adj[i]` 存储课程 `i` 指向的所有后续课程 (与 BFS 方法相同, 但注意边的方向定义, 这里 `[ai, bi]` 仍表示 `bi -> ai`)。

2. 初始化状态数组:

- 创建一个状态数组 `visited`, 大小为 `numCourses`, 所有元素初始化为 0 (未访问)。

3. 遍历所有节点:

- 对每个课程 `i` 从 0 到 `numCourses - 1`:
 - 如果 `visited[i] == 0` (未访问), 则调用 `dfs(i)` 进行深度优先搜索。
 - 如果 `dfs(i)` 返回 `false` (表示在以 `i` 为起点的搜索中发现了环), 则直接返回 `false`。

4. DFS 函数 `dfs(u)`:

- **功能:** 从节点 `u` 开始进行 DFS, 检测是否存在环。返回 `true` 表示无环, `false` 表示有环。
- **标记为正在访问:** `visited[u] = 1`。
- **遍历邻居:** 遍历 `u` 的所有后续课程 `v` (即 `adj[u]` 中的所有课程):
 - **如果 `v` 未访问 (`visited[v] == 0`):**
 - 递归调用 `dfs(v)`。
 - 如果 `dfs(v)` 返回 `false` (在子路径中发现了环), 则直接返回 `false`。
 - **如果 `v` 正在访问 (`visited[v] == 1`):**
 - 这表示我们通过一条路径又回到了当前 DFS 路径上的一个节点, 形成了环。直接返回 `false`。
 - **如果 `v` 已访问 (`visited[v] == 2`):**
 - 说明从 `v` 出发的路径已经探索过且无环, 无需再次访问, 继续检查下一个邻居。
- **标记为已访问:** 如果 `u` 的所有邻居都处理完毕且未发现环, 则将 `u` 的状态标记为已访问: `visited[u] = 2`。
- **返回无环:** 返回 `true`。

5. **返回结果**：如果所有节点的 DFS 都成功完成（没有返回 `false`），则说明图中无环，返回 `true`。

3. 代码实现 (Python)

```
1  from typing import List
2  import collections
3
4  class Solution:
5      # 方法一：拓扑排序 (BFS / Kahn 算法)
6      def canFinishBFS(self, numCourses: int, prerequisites: List[List[int]])
7      -> bool:
8          """
9          使用 BFS (Kahn 算法) 进行拓扑排序来判断课程是否能完成。
10         Args:
11             numCourses: 课程总数。
12             prerequisites: 先修课程列表, [[ai, bi]] 表示学习 ai 必须先学习 bi。
13         Returns:
14             如果可以完成所有课程则返回 True, 否则返回 False。
15         """
16         # 1. 构建邻接表和入度数组
17         adj = collections.defaultdict(list) # 邻接表, 存储 bi -> [ai1, ai2, ...]
18         in_degree = [0] * numCourses      # 入度数组, in_degree[i] 表示课程 i
19         的先修课数量
20
21         for ai, bi in prerequisites:
22             adj[bi].append(ai) # 添加从 bi 到 ai 的边
23             in_degree[ai] += 1 # 课程 ai 的入度加 1
24
25         # 2. 初始化队列, 加入所有入度为 0 的课程
26         queue = collections.deque()
27         for i in range(numCourses):
28             if in_degree[i] == 0:
29                 queue.append(i)
30
31         # 3. BFS 过程
32         count = 0 # 记录已完成 (拓扑排序) 的课程数量
33         while queue:
34             u = queue.popleft() # 取出一门可以学习的课程 u
35             count += 1         # 完成课程数加 1
36
37             # 遍历 u 的所有后续课程 v
38             for v in adj[u]:
39                 in_degree[v] -= 1 # 将 v 的入度减 1 (因为 u 已经学完)
40                 # 如果 v 的入度变为 0, 说明 v 的所有先修课都已完成
41                 if in_degree[v] == 0:
42                     queue.append(v) # 将 v 加入队列
43
44         # 4. 结果判断
45         # 如果完成的课程数等于总课程数, 说明无环, 可以完成
46         return count == numCourses
47
48     # 方法二：深度优先搜索 (DFS) 检测环
```

```

47     def canFinishDFS(self, numCourses: int, prerequisites: List[List[int]])
-> bool:
48         """
49         使用 DFS 检测图中是否存在环来判断课程是否能完成。
50         Args:
51             numCourses: 课程总数。
52             prerequisites: 先修课程列表, [[ai, bi]] 表示学习 ai 必须先学习 bi。
53         Returns:
54             如果可以完成所有课程则返回 True, 否则返回 False。
55         """
56         # 1. 构建邻接表
57         adj = collections.defaultdict(list) # 邻接表, 存储 bi -> [ai1, ai2,
...
58         for ai, bi in prerequisites:
59             adj[bi].append(ai)
60
61         # 2. 初始化状态数组
62         # 0: 未访问, 1: 正在访问 (当前递归栈中), 2: 已访问 (安全)
63         visited = [0] * numCourses
64
65         def dfs(u: int) -> bool:
66             """
67             从节点 u 开始进行 DFS, 检测环。
68             Returns:
69                 True 如果从 u 出发无环, False 如果检测到环。
70             """
71             # 如果节点已访问且安全, 直接返回 True
72             if visited[u] == 2:
73                 return True
74             # 如果节点正在被访问 (在当前递归路径上), 说明检测到环
75             if visited[u] == 1:
76                 return False
77
78             # 标记节点 u 为正在访问
79             visited[u] = 1
80
81             # 遍历 u 的所有后续课程 v
82             for v in adj[u]:
83                 # 如果从 v 出发的 DFS 检测到环, 则直接返回 False
84                 if not dfs(v):
85                     return False
86
87             # 如果从 u 出发的所有路径都探索完毕且无环, 标记 u 为已访问 (安全)
88             visited[u] = 2
89             # 返回 True 表示从 u 出发无环
90             return True
91
92         # 3. 遍历所有节点作为起点进行 DFS
93         for i in range(numCourses):
94             # 如果节点 i 未访问过, 则从 i 开始 DFS
95             # 注意: 这里不能用 visited[i] == 0 判断, 因为 dfs 函数内部会修改状态
96             # 应该对每个节点都尝试调用 dfs, dfs 内部会处理已访问状态
97             # 修正: 应该只对未访问过的节点调用 dfs
98             if visited[i] == 0:
99                 if not dfs(i):

```

```

100                                     # 如果任何一次 DFS 调用返回 False (检测到环), 则整体返回
False
101                                     return False
102
103                                     # 如果所有 DFS 都成功完成 (没有检测到环), 则返回 True
104                                     return True
105
106                                     # 最终提交选择其中一种方法
107                                     def canFinish(self, numCourses: int, prerequisites: List[List[int]]) ->
bool:
108                                     return self.canFinishBFS(numCourses, prerequisites)
109                                     # return self.canFinishDFS(numCourses, prerequisites)
110

```

4. 复杂度分析

- 拓扑排序 (BFS):

- 时间复杂度: $O(N + M)$, 其中 N 是课程数 (`numCourses`), M 是先修关系的数目 (`len(prerequisites)`)。构建邻接表和入度数组需要 $O(M)$, BFS 过程中每个节点和每条边最多访问一次, 也是 $O(N + M)$ 。
- 空间复杂度: $O(N + M)$ 。邻接表需要 $O(M)$ 空间, 入度数组需要 $O(N)$ 空间, 队列在最坏情况下可能存储 $O(N)$ 个节点。

- DFS 检测环:

- 时间复杂度: $O(N + M)$ 。每个节点和每条边在 DFS 过程中最多访问一次。
- 空间复杂度: $O(N + M)$ 。邻接表需要 $O(M)$ 空间, `visited` 状态数组需要 $O(N)$ 空间, 递归调用栈的深度最坏情况下为 $O(N)$ 。

5. 如何在面试中讲解

1. 问题抽象:

- "这个问题可以抽象成一个有向图的问题。每门课程是一个节点, 如果课程 `a` 依赖课程 `b`, 就有一条从 `b` 指向 `a` 的有向边。"
- "判断是否能完成所有课程, 等价于判断这个有向图中是否存在环。"

2. 方法一: 拓扑排序 (BFS / Kahn 算法):

- "拓扑排序是检测有向无环图 (DAG) 的经典算法。如果一个图能成功进行拓扑排序, 那它就是无环的。"
- "思路是: 首先找到所有入度为 0 的节点 (没有先修课的课程), 将它们加入队列。"
- "然后, 不断从队列中取出节点 `u`, 表示这门课可以修了。我们将 `u` 视为 '完成', 并更新其所有后续课程 `v` 的入度 (减 1)。"
- "如果某个后续课程 `v` 的入度变为 0, 说明它的所有先修课都已完成, 就将 `v` 加入队列。"
- "重复这个过程, 直到队列为空。"
- "最后, 统计有多少门课程被成功 '完成' (即加入过拓扑序列)。如果数量等于总课程数, 则无环, 返回 `true`; 否则有环, 返回 `false`。"
- "需要构建邻接表和入度数组。时间复杂度 $O(N+M)$, 空间复杂度 $O(N+M)$ 。"

3. 方法二: DFS 检测环:

- "另一种方法是使用深度优先搜索来直接检测环。"
- "我们需要维护每个节点的三种状态：未访问 (0)，正在访问 (1)，已访问 (2)。'正在访问'表示节点在当前的递归调用栈上。"
- "从一个未访问的节点开始 DFS。将其标记为 '正在访问'。"
- "遍历其邻居 `v`："
 - "如果邻居 `v` 是 '正在访问' 状态，说明遇到了反向边，形成了环，返回 `false`。"
 - "如果邻居 `v` 是 '未访问' 状态，递归调用 DFS 访问 `v`。如果递归调用返回 `false`，则也返回 `false`。"
 - "如果邻居 `v` 是 '已访问' 状态，说明从 `v` 出发的路径安全无环，跳过。"
- "如果当前节点的所有邻居都访问完毕且未发现环，将当前节点标记为 '已访问'，并返回 `true`。"
- "我们需要对图中所有未访问过的节点都启动一次 DFS。"
- "时间复杂度 $O(N+M)$ ，空间复杂度 $O(N+M)$ 。"

4. 对比与选择：

- "两种方法都可以解决问题，复杂度也相同。"
- "BFS (拓扑排序) 的思路可能更符合课程学习的直观过程。"
- "DFS 检测环的实现在递归和状态管理上需要更小心。"